UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE TONALÁ (CUTONALÁ)
DIVISIÓN DE INGENIERÍAS E INNOVACIÓN TECNOLÓGICA
LICENCIATURA EN INGENIERÍA EN COMPUTACIÓN

PROYECTO INTEGRADOR DE DESARROLLO DE SOFTWARE • MOMENTO 1

Práctica Momento 1: Aplicar la Metodología DevOps en un Proyecto de Software

Fase de Planificación: Arquitectura, Alcance, Roles, Plan de Trabajo y Herramientas

PROYECTO ASIGNADO — EQUIPO 2

Nombre Oficial: Gestión Web Segura para Cafetería Orgánica y Comercial
Nombre de la Solución: Monchis Café (Plataforma Web & POS Segura con ODS 12)
Enfoque Específico: Priorización de Seguridad Web, RBAC, HTTPS y Reducción de Vulnerabilidades

INTEGRANTES DEL EQUIPO Y ASIGNACIÓN DE ROLES

1. **Ernesto Hatuey Fierro Medina** (Cód: 215487963)	**Scrum Master & DevOps Lead (Ops)**
2. **Jazmín Díaz**	**Frontend Core Developer (Dev)**
3. **Héctor Hernández**	**Backend & Data Core Developer (Dev)**
4. **Oswaldo Millán**	**QA, Test Automation & Security Engineer (QA)**

Docente Titular: Dr. Gabriel Navarro Salcedo
Cátedra: Productos Integradores de Desarrollo de Software
Fecha de Entrega Oficial: 3 de noviembre de 2026 • 23:59 hrs

1. Definición del Proyecto y su ODS Asociado

1.1 Nombre y Problemática del Negocio

El presente documento corresponde al entregable oficial del **Momento 1 (Planificación)** desarrollado por el **Equipo 2** para el proyecto denominado formalmente **"Gestión Web Segura para Cafetería Orgánica y Comercial"**, instrumentado a través de la plataforma web integral y sistema de punto de venta **"Monchis Café"**.

Problemática Detectada: En el sector gastronómico de especialidad, los establecimientos que comercializan simultáneamente café orgánico regional (granos de sombra cultivados de manera agroecológica por pequeños productores) y productos comerciales complementarios se enfrentan a tres barreras operativas críticas:

- **Pérdida de Trazabilidad y Mermas de Café Orgánico:** Inexistencia de mecanismos digitales que registren el origen del grano (finca cafetalera, cooperativa comunitaria, altitud de siembra, método de beneficio y fecha de tostado). Asimismo, la falta de una rotación controlada bajo la política **PEPS (Primeras Entradas, Primeras Salidas)** produce mermas y desperdicio innecesario de insumos orgánicos de alto valor perecedero.
- **Vulnerabilidades en la Seguridad Web y el Manejo de Fondos:** Los sistemas de caja comúnmente utilizados en pequeñas cafeterías operan con contraseñas en texto plano, sesiones compartidas sin caducidad y ausencia de control de acceso basado en roles (RBAC). Esta falta de blindaje expone el negocio al robo hormiga, modificaciones no autorizadas de inventarios y ataques web básicos (inyecciones SQL, secuestro de sesión y scripts maliciosos).
- **Carencia de Fidelización Vinculada a Prácticas Sostenibles:** Falta de incentivos medibles para que los consumidores adopten hábitos ecológicos, tales como llevar su propio termo o taza reutilizable, lo que propicia el uso desmedido de recipientes desechables de polipapel y plásticos no biodegradables.

1.2 Alcance y Arquitectura de la Solución Propuesta

La solución comprende una plataforma web integral bajo arquitectura de monorepo moderno que unifica el catálogo público, el punto de venta táctil (POS) y el panel administrativo seguro:

- **Frontend Reactivo y Punto de Venta (Vue 3 + Vite):** Interfaz táctil ultrarrápida diseñada para tabletas y terminales de mostrador, con guards de navegación que ocultan o habilitan módulos según el rol verificado.
- **Backend API RESTful Robusto (Node.js + Express):** Microservicios ligeros con validación de entradas mediante esquemas Zod, sanitización de datos y respuestas estandarizadas en JSON.
- **Seguridad Web Reforzada (DevSecOps):** Autenticación stateless mediante tokens JWT de corta duración, Refresh Tokens rotativos almacenados exclusivamente en cookies con directivas `httpOnly`, `Secure` y `SameSite=Strict`; control de acceso RBAC con tres niveles (`admin`, `cajero`, `cliente`), 2FA obligatorio para administradores y blindaje con Google reCAPTCHA v2/v3 en accesos públicos.
- **Persistencia Transaccional ACID (PostgreSQL + Prisma ORM):** Modelado de datos normalizado con transacciones atómicas para cobros de caja e integridad referencial en inventarios orgánicos.
- **Orquestación Asíncrona de Reversas (RabbitMQ + Patrón Saga):** Transacciones distribuidas para compensar cobros: si un pedido es cobrado pero los insumos orgánicos se agotaron en el inventario simultáneamente, el patrón Saga emite automáticamente la reversa financiera y la cancelación del pedido.
- **Módulo de Fidelización Ecológica "Monchis Rewards":** Sellos digitales en monedero virtual y bonificación automática por consumo responsable.

1.3 Alineación con los Objetivos de Desarrollo Sostenible (ODS de la ONU)

El proyecto asume como eje rector el **ODS 12: Producción y Consumo Responsables**, incidiendo de forma puntual en las siguientes metas oficiales:

- **Meta 12.2 (Uso eficiente y sostenible de los recursos naturales):** Se impulsa la venta y consumo de café orgánico de especialidad producido bajo sistemas agroforestales de sombra en cooperativas agrícolas de Michoacán, Chiapas y Oaxaca, eliminando la dependencia de fertilizantes sintéticos y fomentando la biodiversidad.
- **Meta 12.5 (Reducción considerable de la generación de desechos mediante prevención, reducción y reutilización):** El sistema implementa la *"Bonificación Ecológica"*: al procesar un cobro en el POS y registrar que el cliente porta su propia taza o termo reutilizable, el software aplica de forma obligatoria un descuento directo de $5.00 MXN e incrementa sellos digitales de lealtad. Esta automatización reduce directamente el desecho de miles de vasos de polipapel al año.
- **Meta 12.3 (Reducción del desperdicio de alimentos):** Mediante el control de inventario PEPS y alertas de caducidad temprana vía correo electrónico, se evita la descomposición y merma de insumos perecederos.

ODS Complementarios: ODS 8 (Trabajo Decente y Crecimiento Económico) mediante la digitalización formal y segura de MiPyMEs; y **ODS 9 (Industria, Innovación e Infraestructura)** mediante la adopción de una infraestructura resiliente de microservicios contenerizados con Docker y CI/CD.

2. Roles del Equipo DevOps (Dev, Ops, QA, Scrum Master)

La metodología DevOps exige disolver la tradicional barrera divisoria entre el desarrollo de software y las operaciones de TI. En virtud de ello, el **Equipo 2** ha estructurado las funciones de sus 4 integrantes asignando responsabilidades específicas que garantizan un flujo continuo de entregas seguras:

Rol Asignado	Integrante	Responsabilidades en el Ciclo DevOps	Herramientas Principales
Scrum Master *(Facilitador Ágil & Flujo de Valor)*	**Ernesto Hatuey Fierro Medina**	• Conducción de ceremonias ágiles: Sprint Planning, Dailies de 15 min, Sprint Reviews y Retrospectivas semanales. • Gestión y auditoría del tablero Kanban en Trello, estableciendo y monitoreando límites de trabajo en progreso (WIP). • Remoción de cuellos de botella e impedimentos técnicos entre desarrollo, calidad y operaciones. • Verificación estricta de la Definición de Terminado (DoD), asegurando que ninguna tarea avance a producción sin pruebas y análisis de seguridad aprobados.	• Trello • GitHub Projects / Milestones • Slack / Discord
Ops *(DevOps & Cloud Engineer)*	**Ernesto Hatuey Fierro Medina**	• Empaquetamiento de la solución en contenedores inmutables mediante `Dockerfile` multi-stage. • Orquestación de entornos locales y de pruebas mediante `docker-compose.yml` (API, PostgreSQL, RabbitMQ). • Configuración y mantenimiento del servidor de CI/CD (Jenkins y GitHub Actions). • Automatización del despliegue: Frontend en Vercel con HTTPS forzado y Backend en contenedor cloud con Nginx Reverse Proxy y TLS. • Configuración de observabilidad: logs estructurados con Winston/Morgan y endpoint `/health`.	• Docker & Docker Compose • Jenkins • GitHub Actions • Vercel (HTTPS SSL) • Nginx / Linux • Winston Logger
Dev (Backend) *(Backend & Data Core Developer)*	**Héctor Hernández**	• Construcción de la API RESTful en Node.js y TypeScript con validaciones estrictas vía Zod. • Modelado y migraciones de esquemas en PostgreSQL mediante Prisma ORM. • Implementación de seguridad: Autenticación JWT stateless con Refresh Tokens en cookies `httpOnly`, hashing `bcrypt` (salt 12) y middleware RBAC (`admin`, `cajero`, `cliente`). • Integración del broker RabbitMQ con patrón Saga para reversas automáticas de cobros ante falta de inventario orgánico.	• Node.js & Express • TypeScript • PostgreSQL & Prisma • RabbitMQ (AMQP) • JWT & bcrypt
Dev (Frontend) *(Frontend Core Developer)*	**Jazmín Díaz**	• Desarrollo de la Single Page Application (SPA) en Vue 3 con Vite y gestión de estado con Pinia. • Construcción del Punto de Venta (POS) táctil optimizado para cajeros con selección rápida y cobro multipago. • Implementación de vistas de catálogo diferenciadas para productos comerciales y café orgánico con trazabilidad ODS 12.	• Vue 3 & Vite • Pinia Store • Vue Router (Guards) • Vanilla CSS Tokens • Google reCAPTCHA

Rol Asignado	Integrante	Responsabilidades en el Ciclo DevOps	Herramientas Principales
		• Configuración de Navigation Guards en Vue Router basados en los claims del token JWT. • Integración de Google reCAPTCHA v2/v3 en formularios de acceso y módulo visual "Monchis Rewards".	
QA *(Test Automation & SecOps Engineer)*	**Oswaldo Millán**	• Automatización de pruebas unitarias y de integración sobre controladores de autenticación y lógica de cobro con Vitest y Supertest. • Creación de pruebas End-to-End (E2E) con Playwright simulando flujos reales de venta y validando la restricción de accesos indebidos. • Ejecución de análisis estático de seguridad (SAST) y escaneo de vulnerabilidades en dependencias mediante `npm audit` y Snyk. • Monitoreo, cuantificación y reporte formal de la métrica clave asignada al Equipo 2: **"Cantidad de vulnerabilidades corregidas"**.	• Vitest & Supertest • Playwright E2E • Snyk Open Source • npm audit • OWASP ZAP

3. Plan de Trabajo (4 Semanas, Tareas y Responsables)

El plan de trabajo del proyecto está programado para ejecutarse a lo largo de **4 semanas naturales**, acopladas de manera exacta a los **4 Momentos de evaluación** estipulados en la rúbrica oficial de la materia impartida por el Dr. Gabriel Navarro Salcedo:

Momento / Fechas	Fase DevOps	Tareas Técnicas Detalladas	Responsables	Hito / Entregable
Momento 1: Planificación 28 Oct – 03 Nov (Entrega: 3 nov)	**Plan & Setup**	**T1.1:** Especificación del alcance, arquitectura técnica y justificación de impacto en el ODS 12. **T1.2:** Creación del repositorio en GitHub, configuración de ramas (`main`, `develop`), reglas de protección y commits semánticos. **T1.3:** Creación y estructuración del tablero Kanban en Trello con políticas de transición de columnas y límites WIP. **T1.4:** Definición de la matriz de roles y permisos (RBAC), esquemas Prisma y directrices de seguridad web. **T1.5:** Elaboración del diagrama de flujo del proceso DevOps y redacción del documento entregable Momento 1.	• Ernesto F. • Héctor H. • Jazmín D. • Oswaldo M.	Documento entregable: `Proyecto_Equipo2_Momento1_Planificacion.pdf` Repositorio y tablero Trello activos.
Momento 2: Desarrollo e Integración Continua 04 Nov – 10 Nov (Entrega: 10 nov)	**Code, Build & CI Test**	**T2.1:** Desarrollo de la API Node.js: endpoints CRUD de productos, autenticación JWT, RBAC y transacciones de caja en PostgreSQL. **T2.2:** Desarrollo del frontend Vue 3:	• Héctor H. (Dev) • Jazmín D. (Dev) • Ernesto F. (Ops) • Oswaldo M. (QA)	Documento entregable: `Proyecto_Equipo2_Momento2_Integracion.pdf` Prototipo funcional MVP, pipeline CI operativo y reporte de pruebas.

Momento / Fechas	Fase DevOps	Tareas Técnicas Detalladas	Responsables	Hito / Entregable
		interfaz POS, catálogo segmentado (orgánico vs comercial) y guards de autenticación. **T2.3:** Contenedorización: confección de `Dockerfile` multi-stage y `docker-compose.yml` local. **T2.4:** Configuración del pipeline de CI en Jenkins / GitHub Actions (Linting + Escaneo SAST Snyk + Tests Vitest). **T2.5:** Creación y ejecución de pruebas unitarias automáticas y documentación del reporte de errores corregidos.		
Momento 3: Despliegue y Monitoreo 11 Nov – 17 Nov (Entrega: 17 nov)	**Release, Deploy & Operate**	**T3.1:** Despliegue continuo del frontend Vue 3 en Vercel con certificado SSL/HTTPS forzado. **T3.2:** Despliegue contenerizado del backend API y PostgreSQL en entorno cloud con Nginx Reverse Proxy y TLS. **T3.3:** Inyección de cabeceras de seguridad web HTTP (Helmet, CSP estricto, HSTS y CORS restrictivo). **T3.4:** Configuración de observabilidad: logs estructurados con Winston/Morgan y sondeo periódico del endpoint	• Ernesto F. (Ops) • Oswaldo M. (QA) • Héctor H. (Dev) • Jazmín D. (Dev)	Documento entregable: `Proyecto_Equipo2_Momento3_Despliegue.pdf` URL pública con HTTPS, logs de monitoreo y reporte de vulnerabilidades.

Momento / Fechas	Fase DevOps	Tareas Técnicas Detalladas	Responsables	Hito / Entregable
		`/health`. **T3.5:** Ejecución de escaneos de vulnerabilidades, parcheo de fallos y documentación formal de la métrica: *"Cantidad de vulnerabilidades corregidas"*.		
Momento 4: Retroalimentación y Cierre 18 Nov – 24 Nov (Entrega: 24 nov)	**Monitor, Learn & Feedback**	**T4.1:** Recolección y evaluación de métricas DevOps del proyecto (Lead Time, frecuencia de despliegues, MTTR y cobertura de tests). **T4.2:** Sesión formal de Retrospectiva de equipo: análisis de lecciones aprendidas y evaluación crítica del stack tecnológico empleado. **T4.3:** Cierre y auditoría del tablero Trello, evidenciando el flujo completado de las historias de usuario. **T4.4:** Elaboración del catálogo de propuestas de mejora continua (ej. GitOps con ArgoCD, orquestación Kubernetes, escaneo DAST dinámico). **T4.5:** Integración del documento final de reflexión y cierre de proyecto.	• Ernesto F. (SM) • Todo el equipo (Ernesto, Héctor, Jazmín, Oswaldo)	Documento entregable: `Proyecto_Equipo2_Momento4_Reflexion.pdf` Evidencias finales de Trello y balance reflexivo.

4. Herramientas DevOps Seleccionadas y su Justificación Técnica

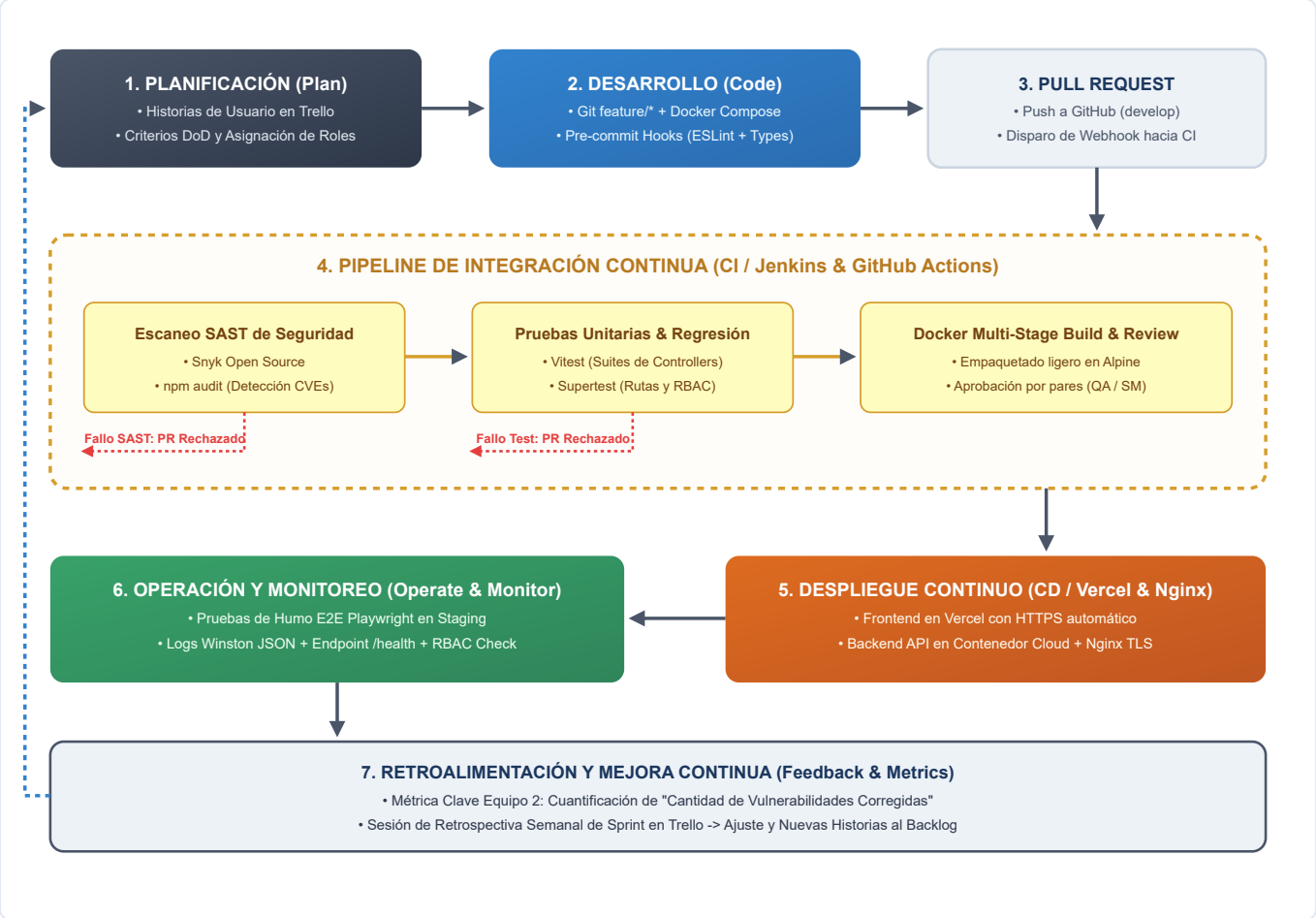
En cumplimiento estricto con las tecnologías sugeridas y directrices específicas del docente para el **Equipo 2**, se seleccionó una cadena de herramientas (Toolchain) de grado profesional:

Categoría DevOps	Herramienta Seleccionada	Justificación Técnica y Rol en el Proyecto
Planificación y Gestión Ágil	**Trello & GitHub Projects**	Tablero Kanban visual para administrar el flujo de valor bajo metodología ágil. Dispone de columnas DevOps estructuradas: *Backlog*, *Sprint To-Do*, *In Progress*, *Code Review & SAST*, *QA Testing*, *Staging Deployed* y *Done*. Permite fijar límites WIP (Work In Progress), asignar miembros y definir checklists con criterios de aceptación (Definition of Done) vinculados a GitHub Issues.
Control de Versiones (SCM)	**GitHub (GitFlow)**	Gestor central de código con estrategia GitFlow adaptada. Se aplican reglas de protección sobre las ramas `main` y `develop` que impiden el push directo y exigen al menos dos revisiones por pares (QA y Scrum Master), sumado a la ejecución exitosa obligatoria del pipeline de CI. Se aplican Conventional Commits para total trazabilidad.
Integración Continua (CI)	**Jenkins & GitHub Actions**	**Jenkins:** Servidor de automatización auto-hospedado para orquestar builds profundos de contenedores y suites completas. **GitHub Actions:** Ejecución automatizada en cada Pull Request para ejecutar en paralelo: verificación de tipos con TypeScript, linting estricto con ESLint, análisis SAST de dependencias con Snyk y pruebas unitarias automáticas con Vitest.
Contenedorización e Infraestructura	**Docker & Docker Compose**	**Docker:** Construcción de imágenes inmutables mediante `Dockerfile` multi-stage (etapa de compilación y etapa ligera de ejecución con Alpine Linux), reduciendo el peso de las imágenes a menos de 150 MB y minimizando la superficie de vulnerabilidades. **Docker Compose:** Orquestación local reproducible de la API Node.js, la base de datos PostgreSQL y RabbitMQ en una red aislada (`monchis_net`), erradicando las diferencias de entorno.
Base de Datos y Persistencia	**PostgreSQL 15 & Prisma ORM**	Motor relacional de alto rendimiento que proporciona aislamiento estricto y cumplimiento de transacciones ACID, imprescindible para la coherencia en los cobros de caja del POS. Prisma ORM provee un tipado estricto extremo y parametrización automática de consultas que elimina vulnerabilidades de inyección SQL (SQLi).
Seguridad Web y DevSecOps (Enfoque Equipo 2)	**npm audit, Snyk, Helmet & reCAPTCHA**	**Snyk & npm audit:** Detección temprana de vulnerabilidades (CVEs) en dependencias durante el build, alimentando la métrica oficial: *"Cantidad de vulnerabilidades corregidas"*. **Helmet:** Inyección de cabeceras de protección HTTP (CSP, HSTS, X-Frame-Options, X-Content-Type-Options). **Google reCAPTCHA v2/v3:** Protección en inicio de sesión contra ataques de fuerza bruta y bots.
Pruebas Automatizadas	**Vitest, Supertest & Playwright**	**Vitest & Supertest:** Ejecución ultrarrápida de pruebas unitarias y de integración sobre controladores de autenticación, verificación de permisos RBAC y algoritmos de rotación de inventarios PEPS. **Playwright:** Automatización de pruebas End-to-End (E2E) simulando la navegación real de usuarios para verificar la imposibilidad de que un cajero acceda a funciones administrativas.
Despliegue Continuo (CD) y Hosting	**Vercel (Frontend) & Nginx TLS (Backend)**	**Vercel:** Despliegue automatizado del frontend Vue 3 en CDN global con certificados SSL automáticos y forzado de HTTPS, cumpliendo con la directriz del Momento 3.

Categoría DevOps	Herramienta Seleccionada	Justificación Técnica y Rol en el Proyecto
		Nginx Reverse Proxy: Servidor web para la API Node.js con terminación SSL/TLS segura (puerto 443), compresión gzip y proxy pass hacia el contenedor Docker.
Monitoreo y Observabilidad	**Winston Logger & Endpoint /health**	Sistema de registro estructurado en formato JSON con rotación diaria de logs de auditoría (`access.log` y `error.log`), complementado con sondeo recurrente del estado de salud del backend mediante el endpoint `/health`.

5. Diagrama de Flujo del Proceso DevOps (CI/CD y DevSecOps)

El ciclo de vida del software para la plataforma **Monchis Café** integra la seguridad y la automatización en cada una de sus fases bajo el principio DevSecOps de *"Shift-Left Security"* (anticipación de la seguridad):



5.1 Explicación Paso a Paso de las Etapas del Ciclo:

1. **Planificación y Definición Ágil:** Se definen las historias de usuario en Trello estructuradas con criterios de aceptación estrictos y políticas de seguridad (Definition of Done), asignando tareas entre Devs, QA y Ops.
2. **Codificación y Entornos Locales:** Los desarrolladores crean ramas `feature/*` en Git y desarrollan en sus entornos locales utilizando `docker-compose.yml` para ejecutar réplicas idénticas de PostgreSQL y RabbitMQ. Antes de cada commit, herramientas de pre-commit verifican linting y tipado.
3. **Integración Continua con DevSecOps (Shift-Left):** Cada Pull Request dispara automáticamente el pipeline en GitHub Actions / Jenkins. Primero se somete al análisis estático SAST (Snyk / npm audit); si no existen vulnerabilidades críticas, se ejecutan las suites de pruebas unitarias Vitest. Solo al superar ambas pruebas y contar con aprobación de pares (Code Review) se permite el merge a la rama `develop`.
4. **Despliegue Continuo con HTTPS:** El código integrado se despliega de forma automatizada hacia el entorno de Staging: el frontend se publica en Vercel con certificado SSL forzado, mientras que el backend y la base de datos se despliegan en contenedores cloud protegidos por un proxy inverso Nginx con cifrado TLS/SSL.

5. **Monitoreo, Métrica de Seguridad y Retroalimentación:** En Staging, QA ejecuta pruebas E2E con Playwright y auditorías dinámicas. El sistema registra logs estructurados con Winston y responde en el endpoint `/health`. Los fallos corregidos se cuantifican en la métrica oficial (*vulnerabilidades corregidas*) y se analizan en la retrospectiva semanal de Sprint para alimentar el ciclo siguiente.

6. Conclusiones y Compromiso con el Ciclo DevOps

La formulación del plan del **Momento 1** para **Monchis Café** consolida las bases de un proyecto de ingeniería de software de alto nivel académico y profesional:

- **Seguridad Embebida desde el Origen:** Se superan las prácticas empíricas al integrar el análisis de vulnerabilidades, el control RBAC y la protección HTTPS como requisitos indispensables en cada fase del ciclo de vida.
- **Impacto Tangible en el ODS 12:** La arquitectura técnica respalda directamente los objetivos de desarrollo sostenible al vincular el sistema de caja con la bonificación por reutilización de recipientes y la trazabilidad de café orgánico de sombra de productores locales.
- **Paridad y Eficiencia Operativa:** La contenedorización con Docker y los pipelines de CI/CD garantizan una transición transparente desde la estación de desarrollo hasta el despliegue en la nube, garantizando el cumplimiento de los cuatro momentos del curso.

7. Referencias Bibliográficas (Normas APA 7ma Edición)

Bass, L., Weber, I., & Zhu, L. (2015). *DevOps: A software architect's perspective*. Addison-Wesley Professional.

Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

International Organization for Standardization, & International Electrotechnical Commission. (2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models* (ISO/IEC Standard No. 25010:2011). ISO/IEC.

Kim, G., Humble, J., Debois, P., Willis, J., & Forsgren, N. (2021). *The DevOps handbook: How to create world-class agility, reliability, and security in technology organizations* (2nd ed.). IT Revolution Press.

Martin, R. C. (2018). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Organización de las Naciones Unidas. (2015). *Transformar nuestro mundo: la Agenda 2030 para el Desarrollo Sostenible*. Resolución A/RES/70/1 aprobada por la Asamblea General el 25 de septiembre de 2015. ONU. https://sdgs.un.org/es/goals/goal12

OWASP Foundation. (2021). *OWASP Top 10: 2021 The ten most critical web application security risks*. OWASP. https://owasp.org/Top10/